Intelligenza artificiale

Giorgio Poluzzi · 6 novembre 2026

Logical Agents

1 Agente logico

Un **agente logico** ha due componenti:

- **Knowledge base (KB)**: insieme di frasi scritte in un linguaggio formale; costituisce il contenuto specifico del dominio;
- **Motore inferenziale**: algoritmi generici, indipendenti dal dominio, che ricavano nuove frasi a partire da quelle già presenti.

L'approccio è **dichiarativo**: si comunica all'agente cosa sa e gli si chiede cosa fare (**Tell & Ask**).

Un agente si può descrivere a due livelli:

- **livello di conoscenza**: cosa sa;
- **livello implementativo**: quali strutture dati e algoritmi utilizza.

Il ciclo dell'agente

A ogni passo l'agente ripete lo stesso ciclo:

1. **Tell**: comunica alla KB la percezione al tempo t;
2. **Ask**: chiede alla KB quale azione compiere;
3. **Tell**: registra nella KB l'azione scelta;
4. incrementa t e restituisce l'azione.

F

Per funzionare, un agente deve saper:

- rappresentare stati e azioni;
- integrare nuove percezioni;
- aggiornare la propria rappresentazione del mondo;
- dedurre proprietà nascoste del mondo;
- dedurre le azioni corrette da eseguire.

2 Logica, modelli, entailment e inferenza

Logica

Linguaggio formale per rappresentare informazioni da cui trarre conclusioni. Ha due ingredienti:

- **sintassi**: quali frasi sono ben formate;
- **semantica**: il significato delle frasi, cioè quando una frase è vera in un certo modello.

Modello

Mondo formalizzato in cui si può valutare se una frase è vera oppure falsa.

- m è un **modello** di α se α è vera in m;
- $M(\alpha)$ è l'insieme di tutti i modelli di α.

Entailment (conseguenza logica)

$\text{KB} \models \alpha$ ("KB implica logicamente α") significa che α è vera in **ogni** modello in cui KB è vera. In termini di insiemi:

$$\text{KB} \models \alpha \iff M(\text{KB}) \subseteq M(\alpha)$$

Esempio

Se $\text{KB} = \{P \wedge Q\}$ e $\alpha = P$, allora $\text{KB} \models \alpha$: in ogni modello in cui $P \wedge Q$ è vera, anche P è vera.

Inferenza

$\text{KB} \vdash_i \alpha$ significa che α può essere derivata dalla KB usando la procedura i.

Perché serve. Le conseguenze di una KB sono tantissime e α è quasi impossibile da indovinare: l'**entailment** stabilisce che α segue dalla KB, l'**inferenza** fornisce il metodo per trovarla.

Una procedura i si valuta con due proprietà:

Proprietà	Significato	Formula
Correttezza	deriva **soltanto** cose vere	se $\text{KB} \vdash_i \alpha$, allora $\text{KB} \models \alpha$
Completezza	deriva **tutto** ciò che è vero	se $\text{KB} \models \alpha$, allora $\text{KB} \vdash_i \alpha$

3 Logica proposizionale

È la logica più semplice.

Sintassi

I simboli $P_1, P_2, \ldots$ sono **frasi atomiche**. Date le frasi S, S_1, S_2, si costruiscono nuove frasi così:

Frase	Nome
$\neg S$	negazione
$S_1 \wedge S_2$	congiunzione (“e”)
$S_1 \vee S_2$	disgiunzione (“o”)
$S_1 \Rightarrow S_2$	implicazione
$S_1 \Leftrightarrow S_2$	bicondizionale (“se e solo se”)

Semantica

Un **modello** assegna vero (V) o falso (F) a ogni simbolo, per esempio $P_{1,2} = F$, $P_{2,2} = F$, $P_{3,1} = V$. Con 3 simboli ci sono $2^3 = 8$ modelli, enumerabili automaticamente.

Regole di valutazione:

- $\neg S$ è vera se S è falsa;
- $S_1 \wedge S_2$ è vera se entrambe sono vere;
- $S_1 \vee S_2$ è vera se almeno una è vera;
- $S_1 \Rightarrow S_2$ è falsa **solo** quando S_1 è vera e S_2 è falsa;
- $S_1 \Leftrightarrow S_2$ è vera quando valgono entrambe le implicazioni ($S_1 \Rightarrow S_2$ e $S_2 \Rightarrow S_1$).

Tavola di verità:

S_1	S_2	$\neg S_1$	$S_1 \wedge S_2$	$S_1 \vee S_2$	$S_1 \Rightarrow S_2$	$S_1 \Leftrightarrow S_2$
F	F	V	F	F	V	V
F	V	V	F	V	V	F
V	F	F	F	V	F	F
V	V	F	V	V	V	V

Esempio di valutazione ricorsiva

Con il modello di sopra:

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = V \wedge (F \vee V) = V \wedge V = V$$

4 Inferenza per enumerazione

Si enumerano i modelli con una **ricerca in profondità** (depth-first search): a ogni passo si fissa un simbolo a vero e poi a falso. KB $\models \alpha$ vale se α è vera in tutti i modelli in cui è vera la KB.

Inference by enumeration 1

```
function TT-Entails?(KB, α) returns true or false
  inputs: KB, knowledge base, a sentence in prop logic
          α, a query, sentence in prop logic

  symbols ← a list of the proposition symbols in KB and α
  return TT-Check-All(KB, α, symbols, [])
```

Inference by enumeration 2

```
function TT-Check-All(KB, α, symbols, model) returns true or false
  if Empty?(symbols) then
    if PL-True?(KB, model) then return PL-True?(α, model)
    else return true
  else do
    P ← First(symbols); rest ← Rest(symbols)
    return TT-Check-All(KB, α, rest, Extend(P, true, model))
       and TT-Check-All(KB, α, rest, Extend(P, false, model))
```

5 Metodi di dimostrazione

Model checking

- enumerazione con tavole di verità
- backtracking (DPLL)
- ricerca locale euristica, min-conflicts/hill-climbing

Applicazione delle regole di inferenza

- si generano frasi nuove sulla base di quelle vecchie
- una dimostrazione è una sequenza di applicazioni di regole
- viene richiesto di portare le frasi in forma normale

6 Equivalenza logica

Definizione

Due frasi sono equivalenti se e solo se sono vere negli stessi modelli, ovvero se una implica l'altra:

$$\alpha \equiv \beta \iff \alpha \models \beta \text{ e } \beta \models \alpha$$

Equivalenza			Nome
$(\alpha \wedge \beta)$	$\equiv$	$(\beta \wedge \alpha)$	commutatività di $\wedge$
$(\alpha \vee \beta)$	$\equiv$	$(\beta \vee \alpha)$	commutatività di $\vee$
$((\alpha \wedge \beta) \wedge \gamma)$	$\equiv$	$(\alpha \wedge (\beta \wedge \gamma))$	associatività di $\wedge$
$((\alpha \vee \beta) \vee \gamma)$	$\equiv$	$(\alpha \vee (\beta \vee \gamma))$	associatività di $\vee$
$\neg(\neg\alpha)$	$\equiv$	α	eliminazione della doppia negazione
$(\alpha \Rightarrow \beta)$	$\equiv$	$(\neg\beta \Rightarrow \neg\alpha)$	contrapposizione
$(\alpha \Rightarrow \beta)$	$\equiv$	$(\neg\alpha \vee \beta)$	eliminazione dell'implicazione
$(\alpha \Leftrightarrow \beta)$	$\equiv$	$((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	eliminazione del bicondizionale
$\neg(\alpha \wedge \beta)$	$\equiv$	$(\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta)$	$\equiv$	$(\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma))$	$\equiv$	$((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributività di $\wedge$ su $\vee$
$(\alpha \vee (\beta \wedge \gamma))$	$\equiv$	$((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributività di $\vee$ su $\wedge$

7 Validità & soddisfacibilità

Validità

Una proposizione è valida (tautologia) se è vera in tutti i modelli.

Esempio

$A \vee \neg A$
$A \Rightarrow A$
$(A \wedge (A \Rightarrow B)) \Rightarrow B$

Soddisfacibilità

Una proposizione è soddisfacibile se è vera in almeno un modello.

Esempio

$A \vee B$
C

Insoddisfacibile

Una proposizione è insoddisfacibile se non è vera in nessun modello.

Esempio

$A \wedge \neg A$

Approccio con l'inferenza

Teorema di deduzione:

- $\text{KB} \models \alpha \iff (\text{KB} \Rightarrow \alpha)$ è valida.

Refutazione:

- $\text{KB} \models \alpha \iff (\text{KB} \wedge \neg\alpha)$ è insoddisfacibile.

Sistema di inferenza

Un sistema di inferenza è un insieme di regole $\mathcal{R}$ della forma:

$$\frac{A_1 \ \dots \ A_k}{A} \quad \text{con} \quad \frac{\text{premesse}}{\text{conclusione}}$$

Si ha una **derivazione** di A da un insieme di formule Γ con un sistema inferenziale $\mathcal{R}$ ($\Gamma \vdash_{\mathcal{R}} A$) se esiste una successione $A_1, \ldots, A_n$ tale che:

- A è A_n;
- per ogni $i \in \{1, \ldots, n\}$ vale una delle due condizioni:
  1. $A_i \in \Gamma$, oppure
  2. A_i è una derivazione diretta di formule che la precedono all'interno della stessa successione.

La successione $A_1, \ldots, A_n$ è una **dimostrazione** di A, e l'insieme di formule Γ viene definito come insieme delle premesse (o assunzioni/ipotesi) per A.

Proprietà dei sistemi di inferenza

- Correttezza: le conclusioni sono sempre conseguenze logiche delle premesse $\psi_1, \ldots, \psi_n \models \psi$.
- Completezza: se $H \models \psi$, esiste allora una derivazione di ψ da H.
- Completezza refutazionale: se $H \cup \{\neg\psi\}$ è insoddisfacibile, ne si riesce a derivare la contraddizione.

Problema di deduzione

Si vuole dimostrare $\Gamma \models A$. Si può quindi procedere per refutazione (bisogna dimostrare che $\Gamma\neg A$ è insoddisfacibile), o per Forward/Backward chaining, ovvero algoritmi polinomiali (funzionano solo su una classe ristretta di formule, ovvero le clausole di Horn).

CNF - Forma Normale Congiuntiva, ovvero una congiunzione ($\wedge$) di clausole, in cui ogni clausole è una disgiunzione ($\vee$) di litterali.

Ogni formula è convetibile in CNF.

Risoluzione

Se una clausola contiene un litterale ℓ e un'altra contiene il suo opposto $\neg\ell$ li elimino entrambi e unisco il resto in una nuova clausola.

Algoritmo di risoluzione

```
function PL-Resolution(KB,α) returns true or false
  inputs: KB,α
  clauses ← set of clauses in CNF form of KB∧¬α
  new ← {}
  loop do
    for each Cᵢ, Cⱼ in clauses do
      resolvents ← PL-Resolve(Cᵢ,Cⱼ)
      if resolvents contains the empty clause then return true
      new ← new ∪ resolvents
    if new ⊆ clauses then return false
    clauses ← clauses ∪ new
```

Forward & Backward chaining

KB = è la congiunzione delle clausole di Horn.

Una clausola di Horn può essere di due tipi:

- un simbolo singolo, ovvero un fatto (A);
- una congiunzione di simboli che implica un simbolo (C $\wedge$ D $\Rightarrow$ B)

Su queste clausole si utilizza la regola del **Modus Ponens** (completa con le clausole di Horn, e gli algoritmi che ne derivano impiegano un tempo di esecuzione lineare): se esistono $\alpha_1, \ldots, \alpha_n$ e $\alpha_1 \wedge \cdots \wedge \alpha_n \Rightarrow \beta$, allora è possibile dedurre β.

Si vuole quindi attivare ogni regola le cui premesse sono già note, aggiungere le conclusioni alla coda della KB e ripetere fino alla risoluzione della query.

Forward Chanining Algorithm

```
function PL-FC-Entails? (KB,q) returns true or false
  inputs: KB, q
  local variables: count, a table indexed by clause
                   inferred, a table indexed by symbol
                   agenda, a list of symbols
  while agenda is not empty do
    p ⟵ Pop(agenda)
    unless inferred[p] do
      inferred[p] ⟵ true
      for each Horn Clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if Head[c] = q then return true
          Push(Head[c], agenda)
  return false
```